DEPLOYMENT AND OPERATIONAL RUNBOOK

CampusConnect: University Club, Event & Activity Management Platform

Attribute	Specification Detail
Document Identifier	CC-OPS-2026-V1.0 (IEEE Std 828-2012 Standard)
Engineering Version	Version 1.0 (Operational Baseline)
Software Engineering Team	Md. Shakhawat Hossain (Student ID: 0432320005101009) | Fahim Hossain (Student ID: 0432320005101007) | Department of CSE
Course Advisor & Supervisor	Al Imtiaz | Department of Computer Science and Engineering
Reviewing Body	Faculty Academic Review Board | Division of Computing and Systems
Release Date & Status	October 2026 | Approved for Engineering Implementation

1. Introduction and System Overview

1.1 Purpose

This Deployment and Operational Runbook establishes standard operating procedures for installing, configuring, deploying, monitoring, and maintaining CampusConnect, a centralized university extracurricular and club management platform. It serves as the operational guide for software engineers, DevOps personnel, and campus systems administrators.

1.2 System Topology

CampusConnect is architected as a modular containerized stack comprising an Express.js/TypeScript REST API, a single-page web frontend, a persistent PostgreSQL 16 database, an in-memory Redis 7 instance, and an Nginx reverse proxy providing TLS termination and static caching.

2. Infrastructure Prerequisites and Environment Matrix

2.1 Hardware and OS Requirements

Component	Minimum Specification	Recommended Production
Compute / CPU	2 Virtual Cores (x86_64 or ARM64)	4 Virtual Cores (Dedicated)
Memory (RAM)	4 GB RAM	8 GB to 16 GB ECC RAM
Disk Storage	20 GB SSD Storage	100 GB NVMe SSD with automated snapshots

Component	Minimum Specification	Recommended Production
Operating System	Ubuntu Linux 22.04 LTS	Ubuntu Linux 24.04 LTS (HWE Kernel)
Network Interface	100 Mbps Ethernet	1 Gbps redundant network interfaces

2.2 Software Runtime Dependencies

Technology	Minimum Supported Version	Purpose in Stack
Node.js	Version 20.x LTS (Iron)	Backend API and worker process execution.
PostgreSQL	Version 16.x Alpine	Persistent ACID relational database core.
Redis	Version 7.x Alpine	Fast session cache, rate-limiting, and queue store.
Docker Engine	Version 24.0+	Containerized infrastructure execution.
Docker Compose	Version 2.20+	Multi-container service orchestration.
Nginx	Version 1.24+	Reverse proxy, SSL/TLS termination, and gzip compression.

3. Environment Variable Configuration Schema

Configure runtime secrets via .env in the project root adhering to the following schema:

```
# Server Runtime
NODE_ENV=production
PORT=5000
API_PREFIX=/api/v1
CLIENT_ORIGIN=https://campusconnect.edu

# PostgreSQL Relational Core
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_DB=campusconnect_db
POSTGRES_USER=campusconnect_user
POSTGRES_PASSWORD=your_secure_db_password_here
POSTGRES_POOL_MIN=4
POSTGRES_POOL_MAX=20

# Redis Cache & Rate Limiting
REDIS_HOST=redis
REDIS_PORT=6379
REDIS_PASSWORD=your_secure_redis_password_here

# Cryptography & JWT
JWT_ACCESS_SECRET=your_super_secret_jwt_access_key_min_64_chars
```

```
JWT_ACCESS_EXPIRES_IN=15m
JWT_REFRESH_SECRET=your_super_secret_jwt_refresh_key_min_64_chars
JWT_REFRESH_EXPIRES_IN=7d
ATTENDANCE_HMAC_SECRET=your_hmac_attendance_signing_secret_key
ATTENDANCE_WINDOW_SECONDS=20
CERTIFICATE_SIGNING_KEY=your_university_certificate_rsa_private_key
```

4. Local Development and Testing Setup

Execute the following sequence to bootstrap the local development environment:

```
# Step 1: Install workspace dependencies
npm install

# Step 2: Start PostgreSQL and Redis infrastructure
docker compose up -d postgres redis

# Step 3: Run database seeding script
npm run seed --workspace=server

# Step 4: Start development servers with live reload
npm run dev
```

5. Health Check and Observability Runbook

Endpoint	HTTP Method	Expected Status	Subsystem Checks
/api/v1/health	GET	HTTP 200 OK	Process uptime, memory RSS, event loop lag.
/api/v1/health/ready	GET	HTTP 200 OK	PostgreSQL pool query (SELECT 1), Redis ping (PONG).

6. Backup, Disaster Recovery, and Point-In-Time Restoration

1. Automated Logical Backups: Nightly cron job executes pg_dump with gzip compression, retaining snapshots for 30 days.

2. Emergency Restoration: In case of catastrophic volume corruption, the restoration playbook drops the corrupted DB, recreates the schema, and streams the latest uncompressed SQL archive into PostgreSQL within 30 minutes (RTO < 30m, RPO < 1h).

7. Incident Response and Troubleshooting Playbook

Incident Scenario	Root Cause	Operational Remediation
DB Connection Saturation	Unclosed async queries or excessive pool size.	Terminate idle connections >5m; tune POSTGRES_POOL_MAX.

Incident Scenario	Root Cause	Operational Remediation
Attendance Clock Drift	NTP desynchronization on host machine.	Restart systemd-timesyncd; verify T-1 grace window.
Redis Memory Exhaustion	Keys missing Time-To-Live expiration.	Apply allkeys-lru eviction policy via redis-cli config.